

Diffusion Language Model

Dooyoung Kim
Natural Language Processing Lab, SKKU

Contents

1. Diffusion

2. Large Language Diffusion Models (LLaDA)

3. TiDAR: Think in Diffusion, Talk in Autoregression

4. CForce: Boosting Parallel Decoding for dLLMs via Consistency Forcing

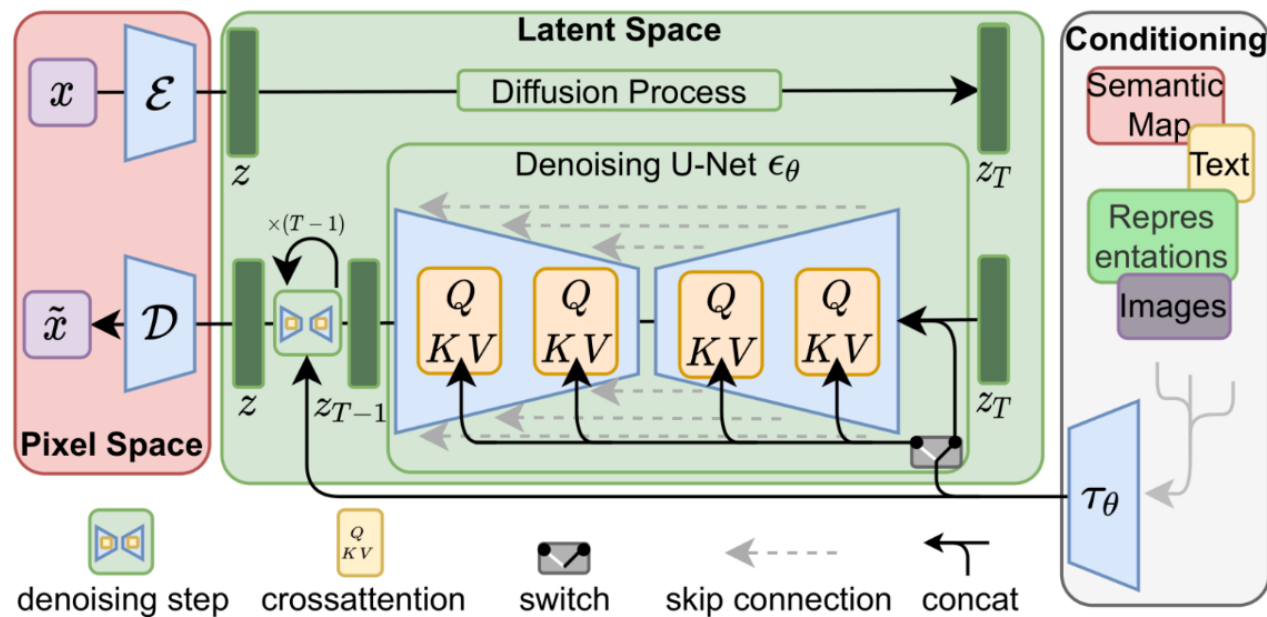
5. Other Topics

What is Diffusion

Diffusion in Computer Vision

Stable Diffusion (2022)

- Random Noise와 Prompt Condition을 기반으로 이미지를 생성
- 여러 번의 Diffusion Step을 통해 denoising을 수행



<Stable Diffusion Architecture >



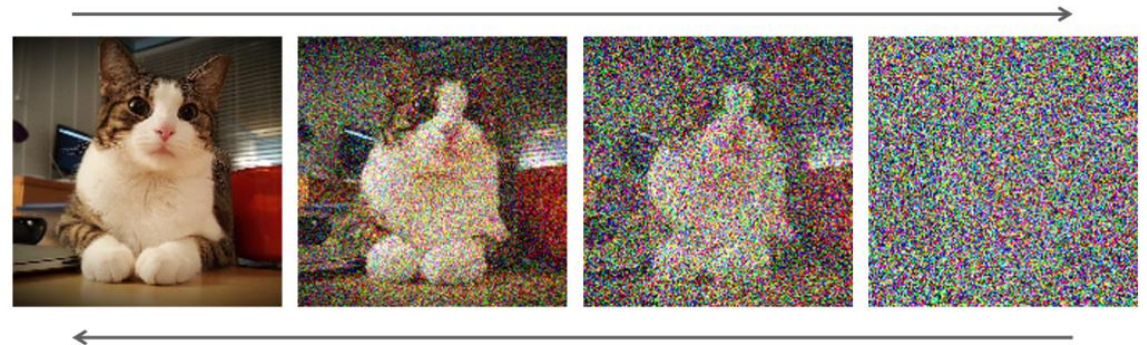
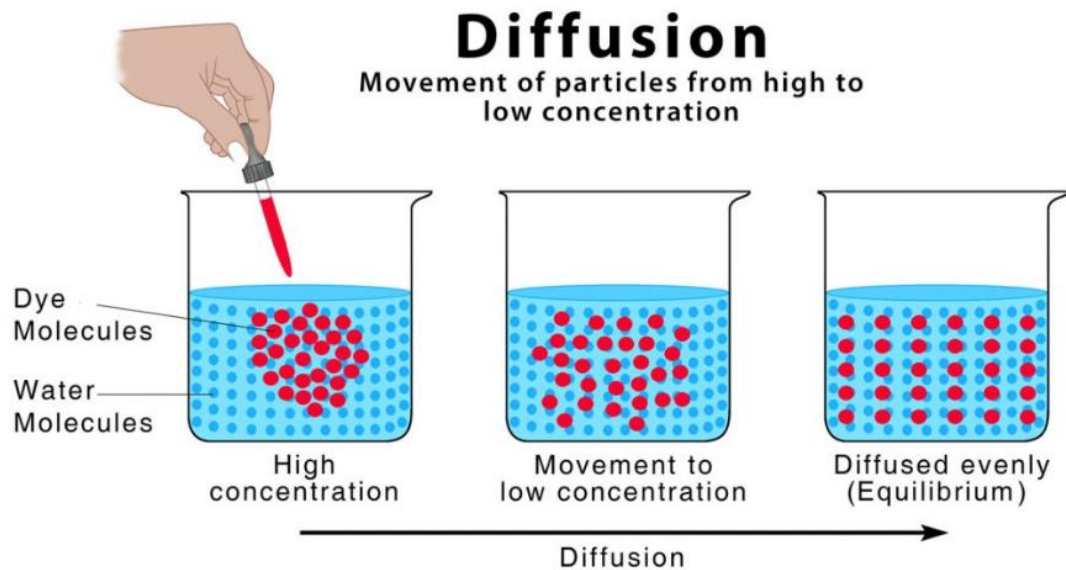
"a photograph of an astronaut riding a horse"

What is Diffusion

Diffusion in Physics

Diffusion

- 시간에 따라 밀집된 입자가 퍼지는 현상
- 열의 이동도 유사한 방식



What is Diffusion

Diffusion Model

Forward Process

- Input 이미지에 Noise를 추가하는 과정
- 원본 이미지 (t=0)에서 Gaussian Noise (t=1)로 변화

$$q(x_{1:T}|x_0) := \prod_{t=1}^T q(x_t|x_{t-1}), \quad q(x_t|x_{t-1}) := \mathcal{N}(x_t; \sqrt{1 - \beta_t}x_{t-1}, \beta_t I)$$

Backward Process

- Input 이미지에서 Noise를 제거하는 과정
- Gaussian Noise (t=1)에서 원본 이미지 (t=0)으로 변화

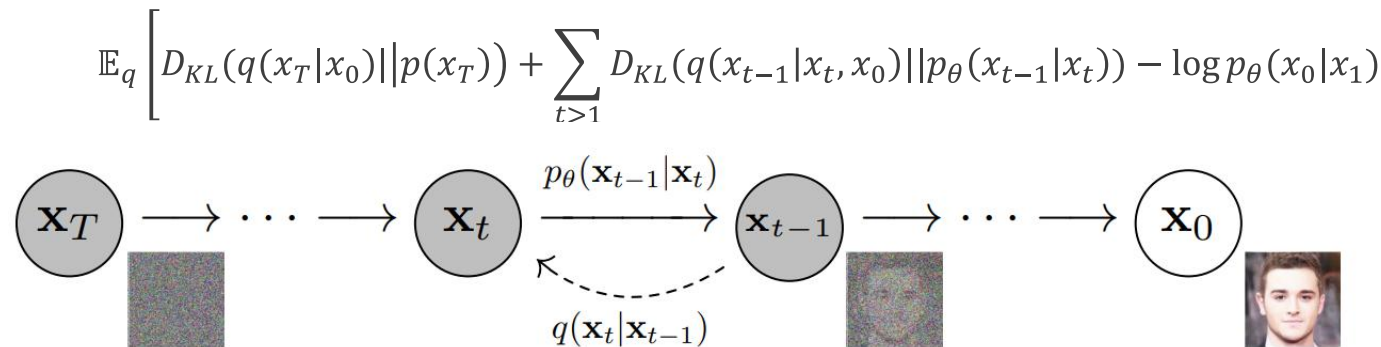


Figure 2: The directed graphical model considered in this work.

Diffusion in NLP

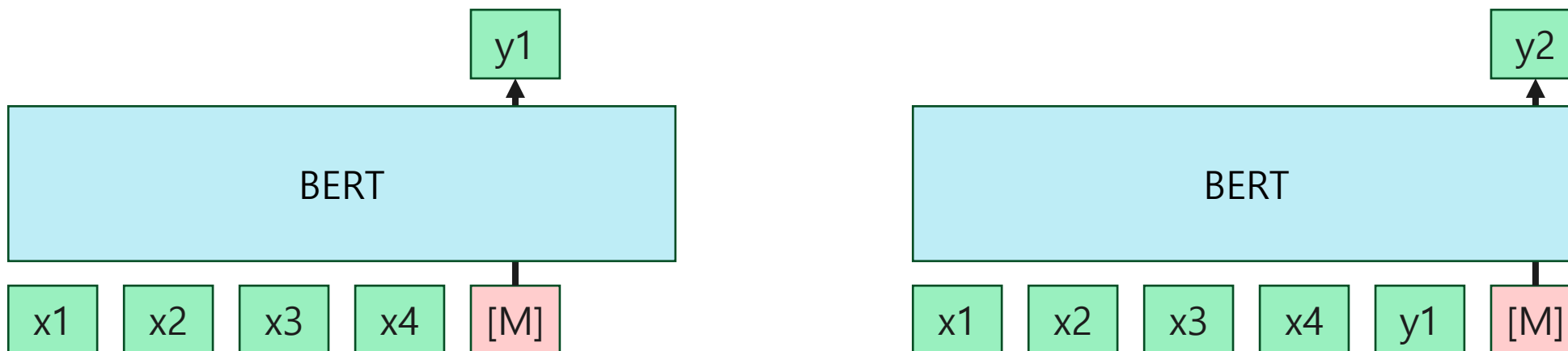
Diffusion Language Model

Continuous Diffusion Language Model

- Language Model의 input embedding, hidden state 등 embedding 단위
- 여러 step으로 embedding을 개선하는 embedding level denoising 수행

Masked Diffusion Language Model

- Clean한 text를 x_0 라 보고, text 내에 [MASK] token을 노이즈로 취급하는 sequence level
- [MASK]로만 이뤄진 시퀀스 x_t 를 기반으로 clean text를 생성
- BERT로 문장을 생성하는 방식과 유사



Diffusion in NLP

Why Diffusion Language Model?

생성 순서의 한계

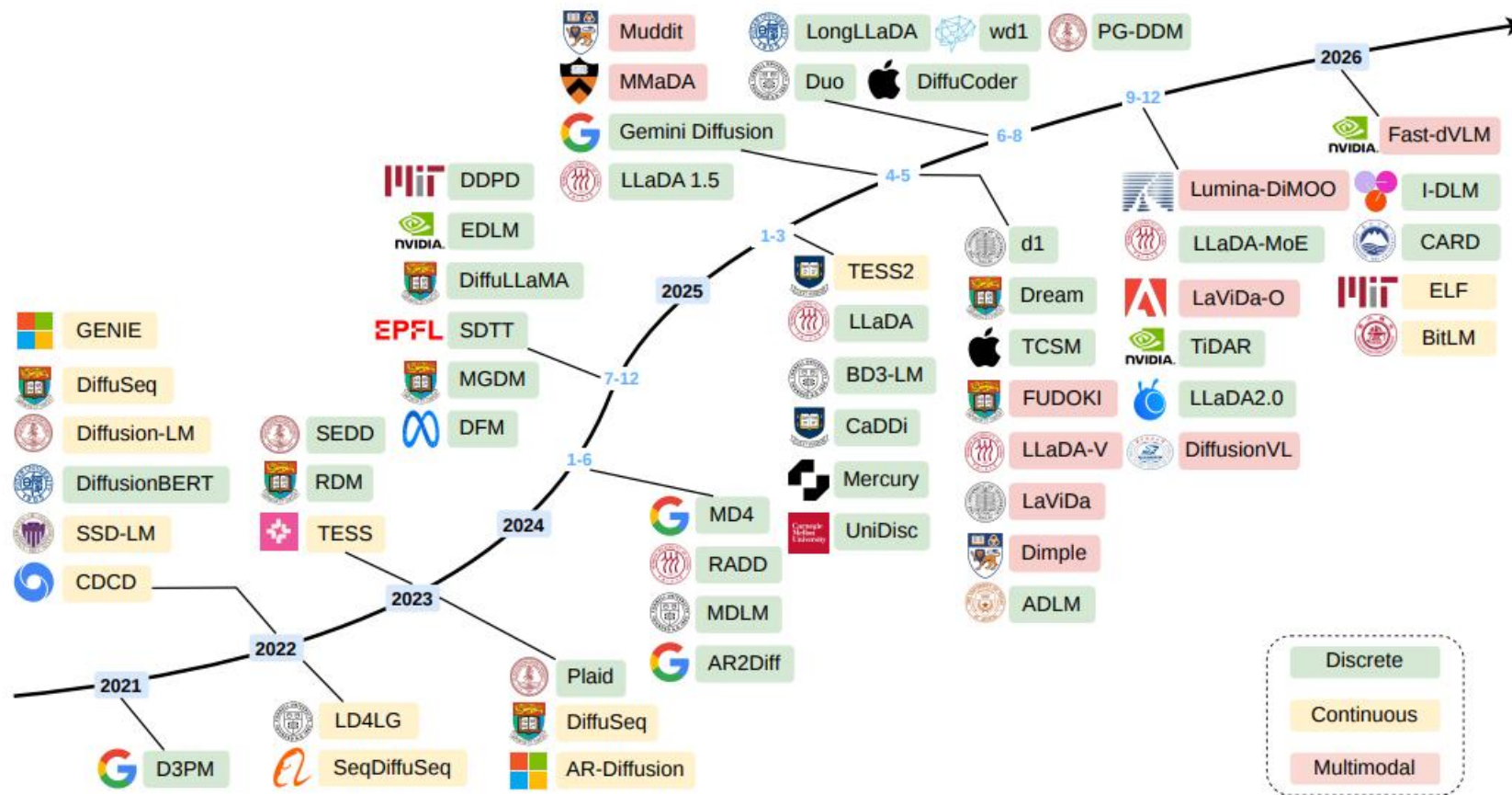
- 기존의 LLM은 Left-to-Right한 순서로 생성
- Left-to-Right 순서로 생성하는 게 항상 optimal 하지 않을 수 있음
- Diffusion Language Model은 생성 순서의 제약 X

생성 효율성

- 기존의 LLM은 auto-regressive한 생성을 수행
- 이전 token이 확정되어야 다음 token을 예측할 수 있음 (1 forward = 1 token)
- 반면, Masked Diffusion Language Model은 [MASK] span으로부터 한번의 forward로 여러 token을 생성

Diffusion in NLP

Recent Papers (2021 ~)



Diffusion in NLP

○ Diffusion Language Models

○ 무엇을 예측하는가

- Current Token: 현재 token을 예측
- Next Token: 다음 token을 예측

○ Denoising 단위가 무엇인가

- Continuous: Embedding-level Denoising
- Discrete: Sentence-level Denoising

○ 어떤 token을 참조하는가

- Causal Attention: 앞에 위치한 token만 attention
- Full Attention: 모든 token과 attention
- Block Diffusion: 앞선 token + Block 내 모든 token과 attention, 뒷 Block의 token은 참조 X

Large Language Diffusion Models

Shen Nie^{1,2,3*†} **Fengqi Zhu**^{1,2,3*†} **Zebin You**^{1,2,3†} **Xiaolu Zhang**^{4‡} **Jingyang Ou**^{1,2,3}
Jun Hu^{4‡} **Jun Zhou**⁴ **Yankai Lin**^{1,2,3‡} **Ji-Rong Wen**^{1,2,3} **Chongxuan Li**^{1,2,3‡§}

¹ Gaoling School of Artificial Intelligence, Renmin University of China

² Beijing Key Laboratory of Research on Large Models and Intelligent Governance

³ Engineering Research Center of Next-Generation Intelligent Search and Recommendation, MOE

⁴ Ant Group

{nieshen, fengqizhu, chongxuanli}@ruc.edu.cn

Large Language Diffusion Models (LLaDA)

○ Motivation

○ Auto Regressive Models (ARMs)

- GPT, LLaMA와 같은 LLM은 Next token prediction 기반의 auto-regressive 방식을 사용
- 이미 auto-regressive model이 scaling, in-context learning 그리고 instruction following을 잘 수행함이 알려짐

○ ARMs limitations

- Left-to-Right 생성 방식에 의한 단방향 의존성
- 역방향 관계를 추론하는 Reverse Curse 등의 일반화 문제

○ Diffusion Language Model

- Image generation에서 Diffusion 기반의 generative model이 성공적임
- Masked Diffusion Model(MDM)을 이용한 문맥 modeling 및 비순차적 token 생성이 가능함
- 잘 정의된 Language Modeling Loss를 기반으로 대규모 MDM을 학습할 수 있지 않을까?

Large Language Diffusion Models (LLaDA)

Overview of LLaDA

LLaDA의 학습 및 추론 방식

(a) Mask all tokens independently

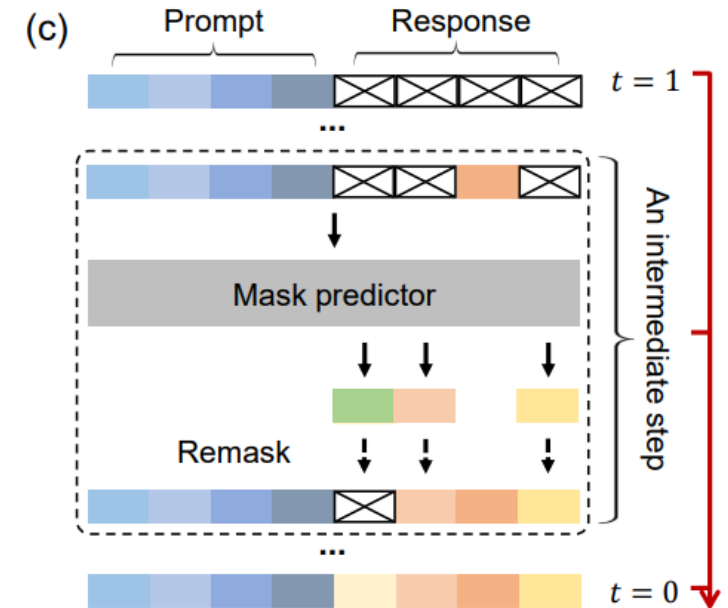
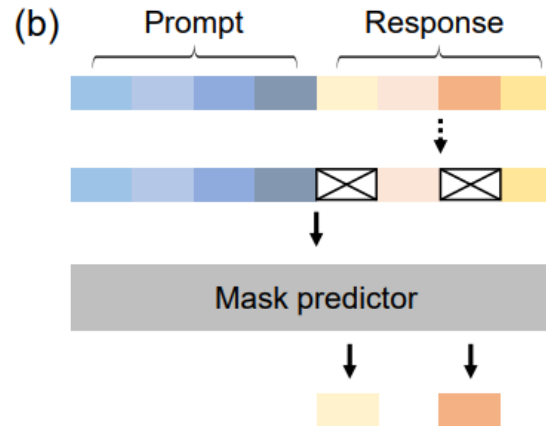
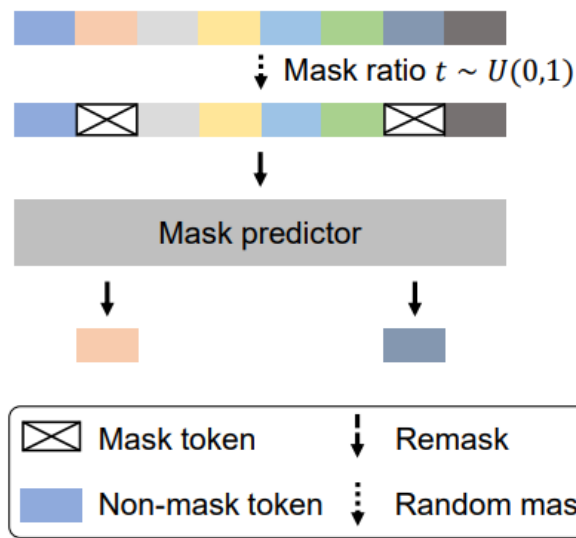


Figure 2: **Overview of LLaDA.** (a) Pre-training. LLaDA is trained on text with random masks applied independently to all tokens at the same ratio $t \sim U[0, 1]$. (b) SFT. Only response tokens are possibly masked. (c) Sampling. LLaDA simulates a diffusion process from $t = 1$ (fully masked) to $t = 0$ (unmasked), predicting all masks simultaneously at each step with flexible remask strategies.

Training LLaDA

LLaDA is a MASK predictor

LLaDA는 MASK token들에 대해서만 cross-entropy loss로 학습된다.

- t 는 $[0, 1]$ 에서 random하게 추출한 continuous random variable (0: clean text, 1: all MASK)
- BERT와 같이 x_t 는 clean text에서 t 비율에 해당하는 token을 MASK로 대체

$$\mathcal{L}(\theta) \triangleq -\mathbb{E}_{t, x_0, t} \left[\frac{1}{t} \sum_{i=1}^L \mathbf{1}[x_t^i = M] \log p_{\theta}(x_0^i | x_t) \right]$$

- LLaDA의 training objective는 negative log-likelihood의 상한선

$$-\mathbb{E}_{p_{data}(x_0)} [\log p_{\theta}(x_0)] \leq \mathcal{L}(\theta)$$

Training LLaDA

○ Theoretical link to maximum likelihood

○ Generative Modeling

- Maximum likelihood는 KL divergence를 최소화 하는 것과 동일함

$$\max_{\theta} \mathbb{E}_{p_{data}(x)} \log p_{\theta}(x) \leftrightarrow \min_{\theta} \text{KL}(p_{data}(x) || p_{\theta}(x))$$

○ MDM의 KL divergence

- $0 \leq s < t \leq 1$ 인 timestep s 와 t 를 가정 (하나의 forward process에서 두 중간 단계 s, t)
- s 에서 MASK token은 t 에서도 MASK

$$P(x_t = M) = t, \quad P(x_s = M) = s$$
$$P(x_s = M | x_t = M) = \frac{s}{t}, \quad P(x_s \neq M | x_t = M) = 1 - \frac{s}{t} = \frac{t-s}{t}$$

Training LLaDA

○ Theoretical link to maximum likelihood

○ Generative Modeling

- Maximum likelihood는 KL divergence를 최소화 하는 것과 동일함

$$\max_{\theta} \mathbb{E}_{p_{data}(x)} \log p_{\theta}(x) \leftrightarrow \min_{\theta} \text{KL}(p_{data}(x) || p_{\theta}(x))$$

○ MDM의 KL divergence

- 임의의 x_t 에 대해서 실제 분포 $q(x_s|x_t)$ 와 모델의 예측 $p_{\theta}(x_s|x_t)$

s/t 확률로 MASK, 그 외에는 clean text

$$q(x_s | x_t = M, x_0) = \frac{s}{t} \delta_M + \frac{t-s}{t} \delta_{x_0}$$

$$p_{\theta}(x_s | x_t = M) = \frac{s}{t} \delta_M + \frac{t-s}{t} p_{\theta}(\cdot | x_t)$$

s/t 확률로 MASK

그 외에는 모델이 예측한 확률 분포

Training LLaDA

○ Theoretical link to maximum likelihood

○ Generative Modeling

- Maximum likelihood는 KL divergence를 최소화 하는 것과 동일함

$$\max_{\theta} \mathbb{E}_{p_{data}(x)} \log p_{\theta}(x) \leftrightarrow \min_{\theta} \text{KL}(p_{data}(x) || p_{\theta}(x))$$

○ MDM의 KL divergence

- 실제 분포 $q(x_s|x_t, x_0)$ 와 모델의 예측 $p_{\theta}(x_s|x_t)$ 의 KL divergence를 계산하면,

$$D_{KL}(q(x_s|x_t, x_0) || p_{\theta}(x_s|x_t)) = \frac{t-s}{t} \log \frac{(t-s)/t}{(t-s)/t p_{\theta}(x_0|x_t)} = \frac{t-s}{t} [-\log p_{\theta}(x_0|x_t)]$$

- $dt = t - s$ 로 정의하고 적분을 취하면,

$$\mathcal{L} = \int_0^1 \mathbb{E}_{x_t \sim x_0} \left[\frac{1}{t} \sum_i \mathbf{1}[x_t^i = M] (-\log p_{\theta}(x_0^i|x_t)) \right] dt \rightarrow \mathcal{L}(\theta) = -\mathbb{E}_{t, x_0, x_t} \left[\frac{1}{t} \sum_i \mathbf{1}[x_t^i = M] \log p_{\theta}(x_0^i|x_t) \right]$$

Monte Carlo Estimate

Training LLaDA

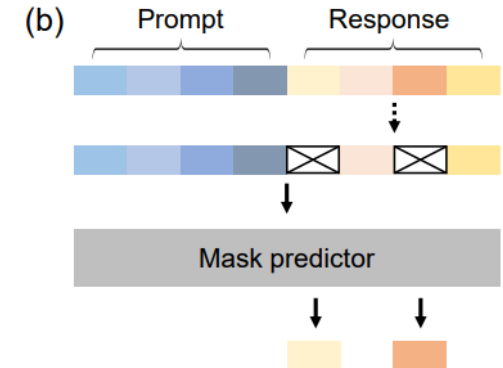
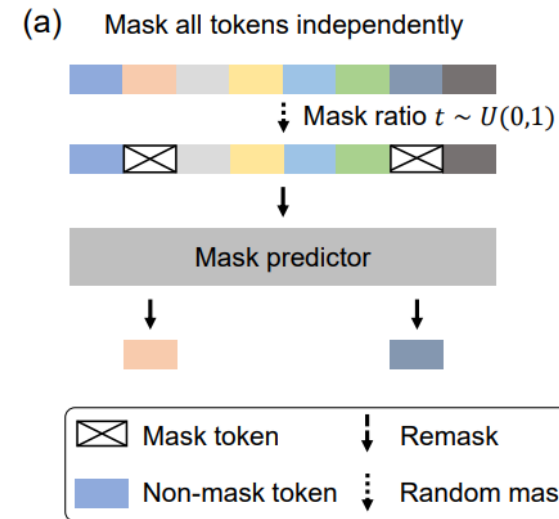
2 Stage Training

Pre-training

- Full attention의 LLaMA 구조 (RMSNorm과 SwiGLU를 사용)
- 1B, 8B 규모의 모델을 2.3T token으로 학습
- 다양한 길이를 학습하기 위해 pre-training data의 1%는 random length를 가지도록 설정
- Warmup-Stable-Decay learning rate scheduler 사용

Supervised Fine-tuning

- Prompt 부분은 clean text로 유지
- 4.5M pair 데이터로 학습
- [EOS] token을 정의하여 max sequence padding



Inference LLaDA

○ Inference

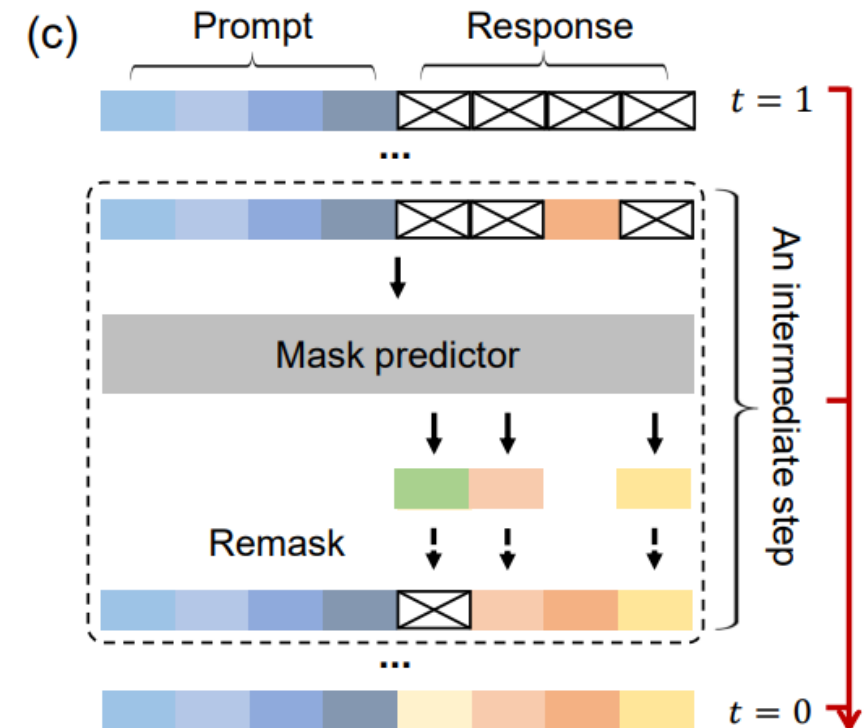
○ Reverse generation process

- Fully masked response로부터 모델 확률 분포를 통해 response를 예측
- LLaDA는 모든 masked token을 동시에 예측하며, 예측된 token 중에서 s/t를 다시 MASK로 되돌림 (가장 confidence가 낮은 token)

○ Conditional likelihood evaluation

- Likelihood evaluation = 특정 답이 생성될 확률
- 실제 모델이 생성한 답이 아니기 때문에 생성 순서를 모름
- 모든 생성 경우의 수를 고려해서 평균값을 활용 (Monte Carlo Estimation)

$$-\mathbb{E}_{l, r_0 r_l} \left[\frac{L}{l} \sum_{i=1}^L \mathbf{1}[r_l^i = M] \log p_{\theta}(r_0^i | p_0, r_l) \right]$$



Experiments

● Benchmark Results of Pre-trained LLMs

	LLaDA 8B*	LLaMA3 8B*	LLaMA2 7B*	Qwen2 7B [†]	Qwen2.5 7B [†]	Mistral 7B [†]	Deepseek 7B [¶]
Model Training tokens	Diffusion 2.3T	AR 15T	AR 2T	AR 7T	AR 18T	AR -	AR 2T
General Tasks							
MMLU	65.9 (5)	65.4 (5)	45.9 (5)	70.3 (5)	74.2 (5)	64.2 (5)	48.2 (5)
BBH	49.7 (3)	62.1 (3)	39.4 (3)	62.3 (3)	70.4 (3)	56.1 (3)	39.5 (3)
ARC-C	45.9 (0)	53.1 (0)	46.3 (0)	60.6 (25)	63.7 (25)	60.0 (25)	48.1 (0)
Hellaswag	70.5 (0)	79.1 (0)	76.0 (0)	80.7 (10)	80.2 (10)	83.3 (10)	75.4 (0)
TruthfulQA	46.1 (0)	44.0 (0)	39.0 (0)	54.2 (0)	56.4 (0)	42.2 (0)	-
WinoGrande	74.8 (5)	77.3 (5)	72.5 (5)	77.0 (5)	75.9 (5)	78.4 (5)	70.5 (0)
PIQA	73.6 (0)	80.6 (0)	79.1 (0)	-	-	-	79.2 (0)
Mathematics & Science							
GSM8K	70.3 (4)	48.7 (4)	13.1 (4)	80.2 (4)	85.4 (4)	36.2 (4)	17.4 (8)
Math	31.4 (4)	16.0 (4)	4.3 (4)	43.5 (4)	49.8 (4)	10.2 (4)	6.0 (4)
GPQA	25.2 (5)	25.9 (5)	25.7 (5)	30.8 (5)	36.4 (5)	24.7 (5)	-
Code							
HumanEval	35.4 (0)	34.8 (0)	12.8 (0)	51.2 (0)	57.9 (0)	29.3 (0)	26.2 (0)
HumanEval-FIM	73.8 (2)	73.3 (2)	26.9 (2)	-	-	-	-
MBPP	40.0 (4)	48.8 (4)	23.2 (4)	64.2 (0)	74.9 (0)	51.1 (0)	39.0 (3)
Chinese							
CMMLU	69.9 (5)	50.7 (5)	32.5 (5)	83.9 (5)	-	-	47.2 (5)
C-Eval	70.5 (5)	51.7 (5)	34.0 (5)	83.2 (5)	-	-	45.0 (5)

Experiments

● Benchmark Results of Post-trained LLMs

	LLaDA 8B*	LLaMA3 8B*	LLaMA2 7B*	Qwen2 7B [†]	Qwen2.5 7B [†]	Gemma2 9B [†]	Deepseek 7B [¶]
Model	Diffusion	AR	AR	AR	AR	AR	AR
Training tokens	2.3T	15T	2T	7T	18T	8T	2T
Post-training	SFT	SFT+RL	SFT+RL	SFT+RL	SFT+RL	SFT+RL	SFT+RL
Alignment pairs	4.5M	-	-	0.5M + -	1M + 0.15M	-	1.5M + -
General Tasks							
MMLU	65.5 (5)	68.4 (5)	44.1 (5)	-	-	-	49.4 (0)
MMLU-pro	37.0 (0)	41.9 (0)	4.6 (0)	44.1 (5)	56.3 (5)	52.1 (5)	-
Hellaswag	74.6 (0)	75.5 (0)	51.5 (0)	-	-	-	68.5 (-)
ARC-C	88.5 (0)	82.4 (0)	57.3 (0)	-	-	-	49.4 (-)
Mathematics & Science							
GSM8K	69.4 (4)	78.3 (4)	29.0 (4)	85.7 (0)	91.6 (0)	76.7 (0)	63.0 (0)
Math	31.9 (0)	29.6 (0)	3.8 (0)	52.9 (0)	75.5 (0)	44.3 (0)	15.8 (0)
GPQA	33.3 (5)	31.9 (5)	28.4 (5)	34.3 (0)	36.4 (0)	32.8 (0)	-
Code							
HumanEval	49.4 (0)	59.8 (0)	16.5 (0)	79.9 (0)	84.8 (0)	68.9 (0)	48.2 (-)
MBPP	41.0 (4)	57.6 (4)	20.6 (4)	67.2 (0)	79.2 (0)	74.9 (0)	35.2 (-)

Experiments

Reversal Reasoning & Example

	Forward	Reversal
GPT-4o (2024-08-06)	82.7	34.3
Qwen2.5-7B Instruct	75.9	38.0
LLaDA-8B Instruct	51.8	45.6

Reversal Curse (시 생성 Task)

- 시의 뒷 구절 (Forward)
- 시의 앞 구절 (Reversal)

(밝은 token이 먼저 생성, 어두운 token이 나중에 생성)

Sampling Process

User

Lily can run 12 kilometers per hour for 4 hours. After that, she runs 6 kilometers per hour.
How many kilometers can she run in 8 hours?

LLaDA

In 4 hours, Lily runs 12 * 4 = 48 kilometers. After 4 hours she runs 6 * 4 = 24 kilometers. In total she runs 48 + 24 = 72 kilometers in 8 hours. Result 72



2025-11-13

TiDAR: Think in Diffusion, Talk in Autoregression

Jingyu Liu^{*1}, Xin Dong^{*}, Zhifan Ye², Rishabh Mehta, Yonggan Fu, Vartika Singh, Jan Kautz, Ce Zhang¹, Pavlo Molchanov

NVIDIA

^{*}Equal Contribution; XD as project lead; Corresponding: jingyu6@uchicago.edu, xind@nvidia.com

TiDAR: Think in Diffusion, Talk in Autoregression

○ Motivation

- Speculative Decoding에 Diffusion을 적용
- Diffusion으로 draft token을 효율적으로 생성, AR로 생성된 draft를 검증

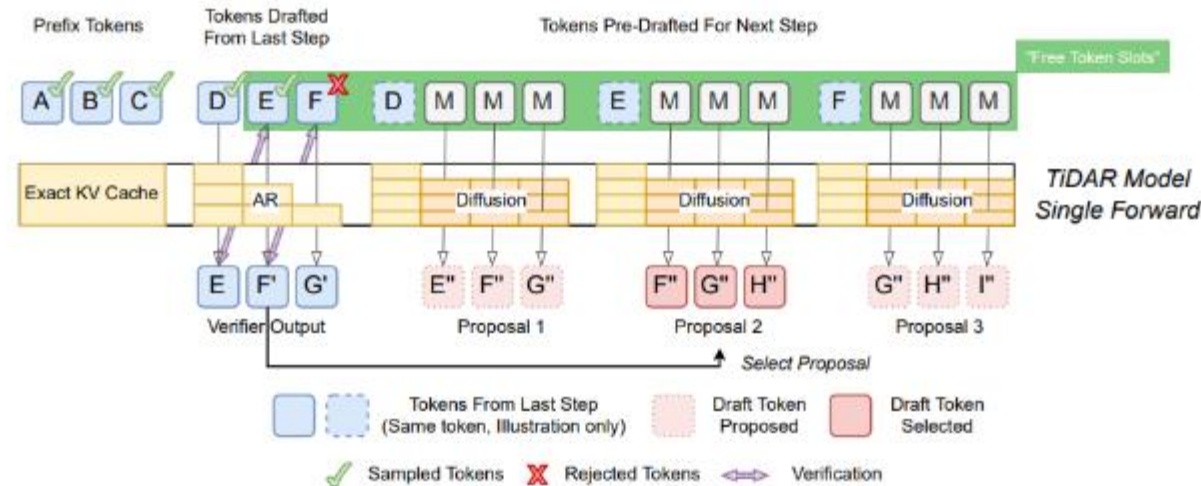


Figure 2 | **TiDAR Architecture:** TiDAR uses a single model forward to sample drafted tokens from the last step and pre-draft tokens for the next step in parallel. By switching the attention pattern among different parts of the sequence, TiDAR encodes the clean tokens drafted from last step causally and mask tokens block-causally (bidirectional within each block) for one-step diffusion pre-drafting. Upon accepting a prefix, the corresponding pre-drafts (proposal) can be selected. The KV cache for tokens forwarded causally will be stored and later evicted if the corresponding tokens are rejected. We illustrate this with a draft length of 3 and an accepted length of 2. Figure 3 shows the exact decoding mask for this example.

TiDAR: Think in Diffusion, Talk in Autoregression

Background

Block Diffusion

- Full Attention을 활용하는 DLM은 KV Cache를 활용하기 어려움
- 또한 고정된 길이 내에서만 응답을 생성할 수 있음
- Block 내에서는 Full Attention을 수행하지만, 뒤 Block은 확인할 수 없는 구조

Autoregression:

✓ High quality

✓ Arbitrary-length

✓ KV caching

✗ Not Parallelizable

Generation steps

There are three categories of the average
 There are three categories of the average rate
 There are three categories of the average rate of...

Diffusion:

✗ Lower quality

✗ Fixed-length

✗ No KV caching

✓ Parallelizable

the reusability will continue to the
 Repeal the reusability cuts and the law will continue to reduce the
 Repeal the reusability cuts and prove the law will continue to reduce the deficit.

Block Diffusion (Ours):

✓ High quality

✓ Arbitrary-length

✓ KV caching

✓ Parallelizable

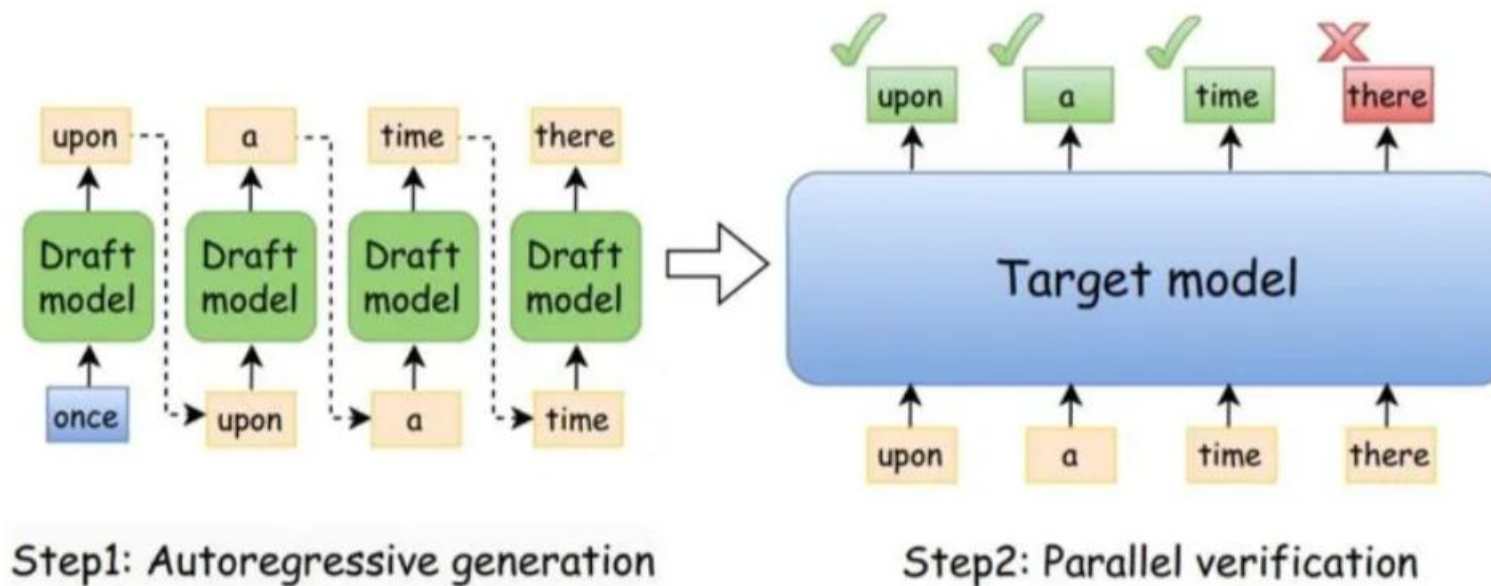
On September 17, we be
 On September 17, 2016, we will be giving the release of
 On September 17, 2016, we will be giving the beta-release of the to our server testing ...

TiDAR: Think in Diffusion, Talk in Autoregression

Background

Speculative Decoding

- N개의 token을 생성하는 것이 검증하는 것보다 더 많은 연산을 필요함 (N vs 1 forward)
- 작은 model로 draft output을 만들고, Target LLM에서 draft를 검증
- Target LLM이 reject한 token부터 다시 작은 model이 draft를 생성



TiDAR: Think in Diffusion, Talk in Autoregression

Overview of TiDAR

- Diffusion과 AR 연산을 하나의 모델에서 수행

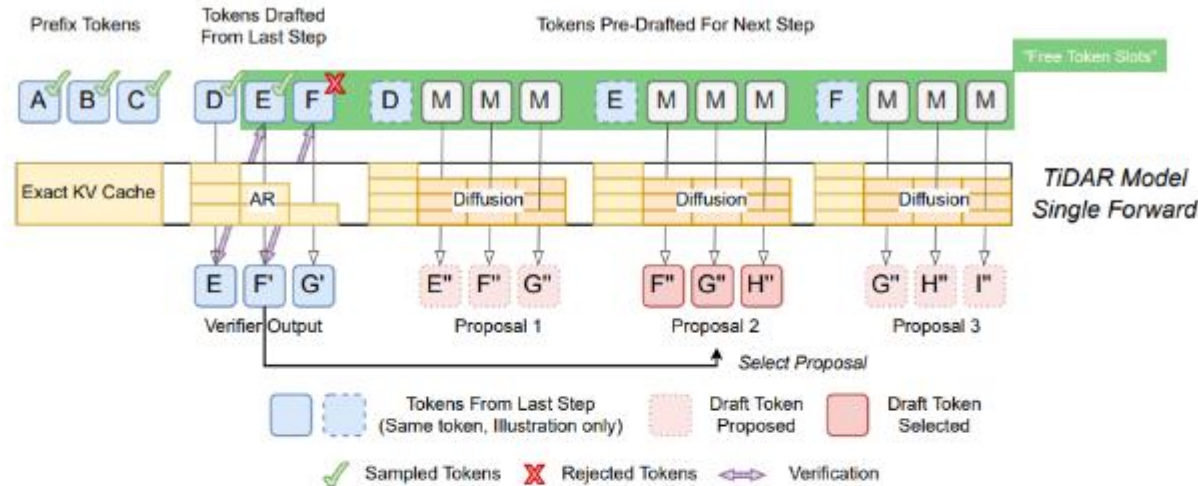


Figure 2 | **TiDAR Architecture:** TiDAR uses a single model forward to sample drafted tokens from the last step and pre-draft tokens for the next step in parallel. By switching the attention pattern among different parts of the sequence, TiDAR encodes the clean tokens drafted from last step causally and mask tokens block-causally (bidirectional within each block) for one-step diffusion pre-drafting. Upon accepting a prefix, the corresponding pre-drafts (proposal) can be selected. The KV cache for tokens forwarded causally will be stored and later evicted if the corresponding tokens are rejected. We illustrate this with a draft length of 3 and an accepted length of 2. Figure 3 shows the exact decoding mask for this example.

TiDAR: Think in Diffusion, Talk in Autoregression

Overview of TiDAR

- 한 번의 forward에서 이전에 생성된 Draft token을 검증하고 동시에 Draft token을 생성

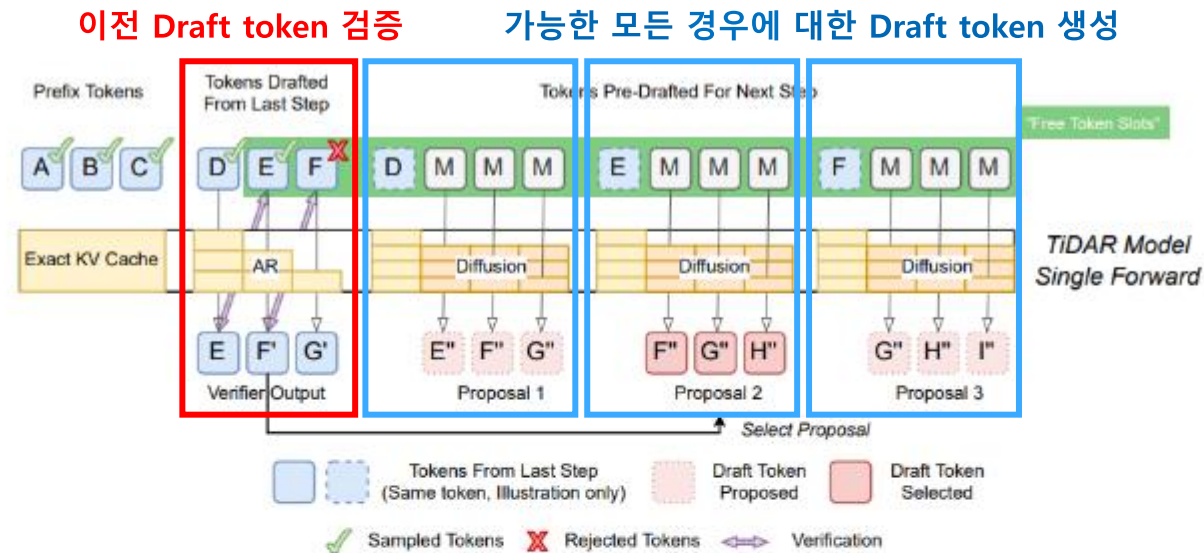


Figure 2 | **TiDAR Architecture:** TiDAR uses a single model forward to sample drafted tokens from the last step and pre-draft tokens for the next step in parallel. By switching the attention pattern among different parts of the sequence, TiDAR encodes the clean tokens drafted from last step causally and mask tokens block-causally (bidirectional within each block) for one-step diffusion pre-drafting. Upon accepting a prefix, the corresponding pre-drafts (proposal) can be selected. The KV cache for tokens forwarded causally will be stored and later evicted if the corresponding tokens are rejected. We illustrate this with a draft length of 3 and an accepted length of 2. Figure 3 shows the exact decoding mask for this example.

Training TiDAR

Diffusion-AR Dual-mode Backbone Training

Backbone Model

- Pretrained LLM 사용 (Qwen2.5-1.5B, Qwen3-4B, Qwen3-8B)
- Block Diffusion의 training recipe를 참고하여 50B token 학습

Prefix: Causal Attention

- Chain-factorized joint distribution을 계산할 수 있음
- Next-token prediction loss를 적용

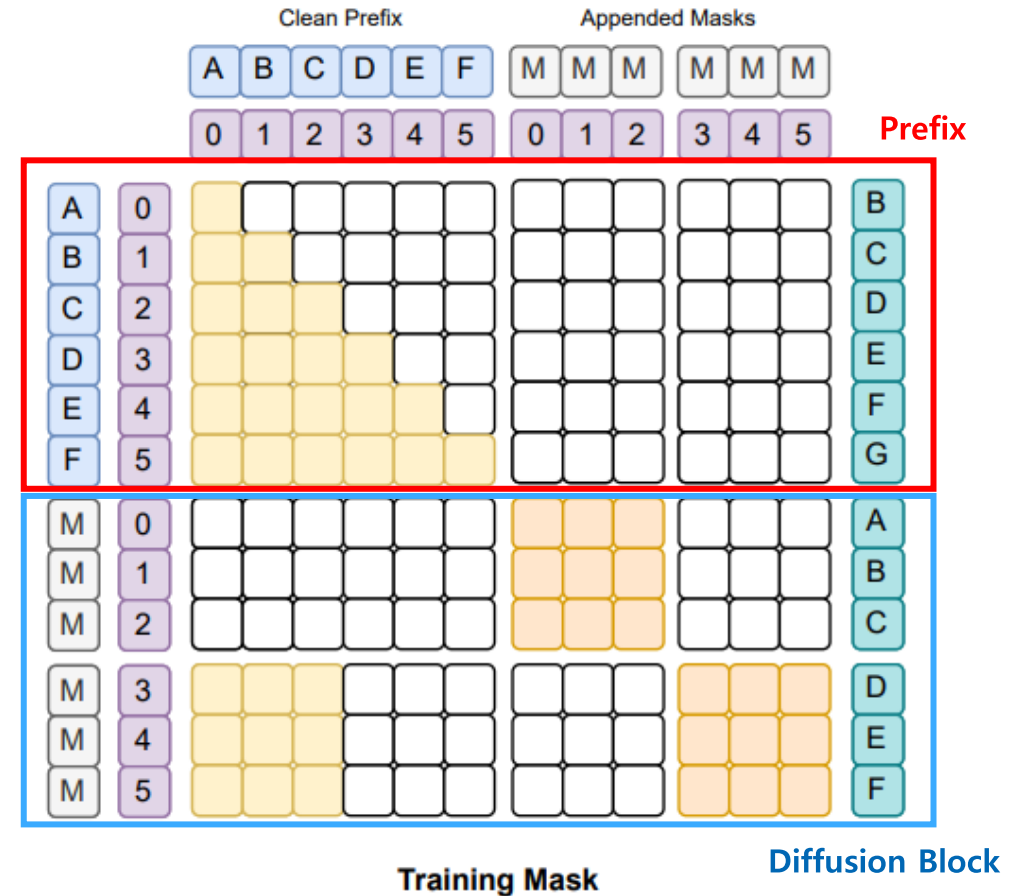
Diffusion Block: Bidirectional Attention

- Block 내의 모든 token을 MASK로 설정
- Full attention을 사용해 MASK token에 Diffusion loss 적용

Training Objective

$$\mathcal{L}_{TiDAR}(\theta) = \frac{1}{1+\alpha} \left(\sum_{i=1}^{S-1} \frac{\alpha}{S-1} \mathcal{L}_{AR}(x_i, x_{i+1}; \theta) + \sum_{i=1}^{S-1} \frac{1}{S-1} \mathcal{L}_{Diff}([mask], x_i; \theta) \right)$$

동일한 Input을 AR mode와 Diffusion mode로 각각 학습



Inference TiDAR

Fully Parallelizable Self-Speculative Generation

Attention Token

- 모든 [MASK] token은 prefix와 block 내 token을 볼 수 있음
- 모든 prefix token은 causal mask를 사용

AR mode verification

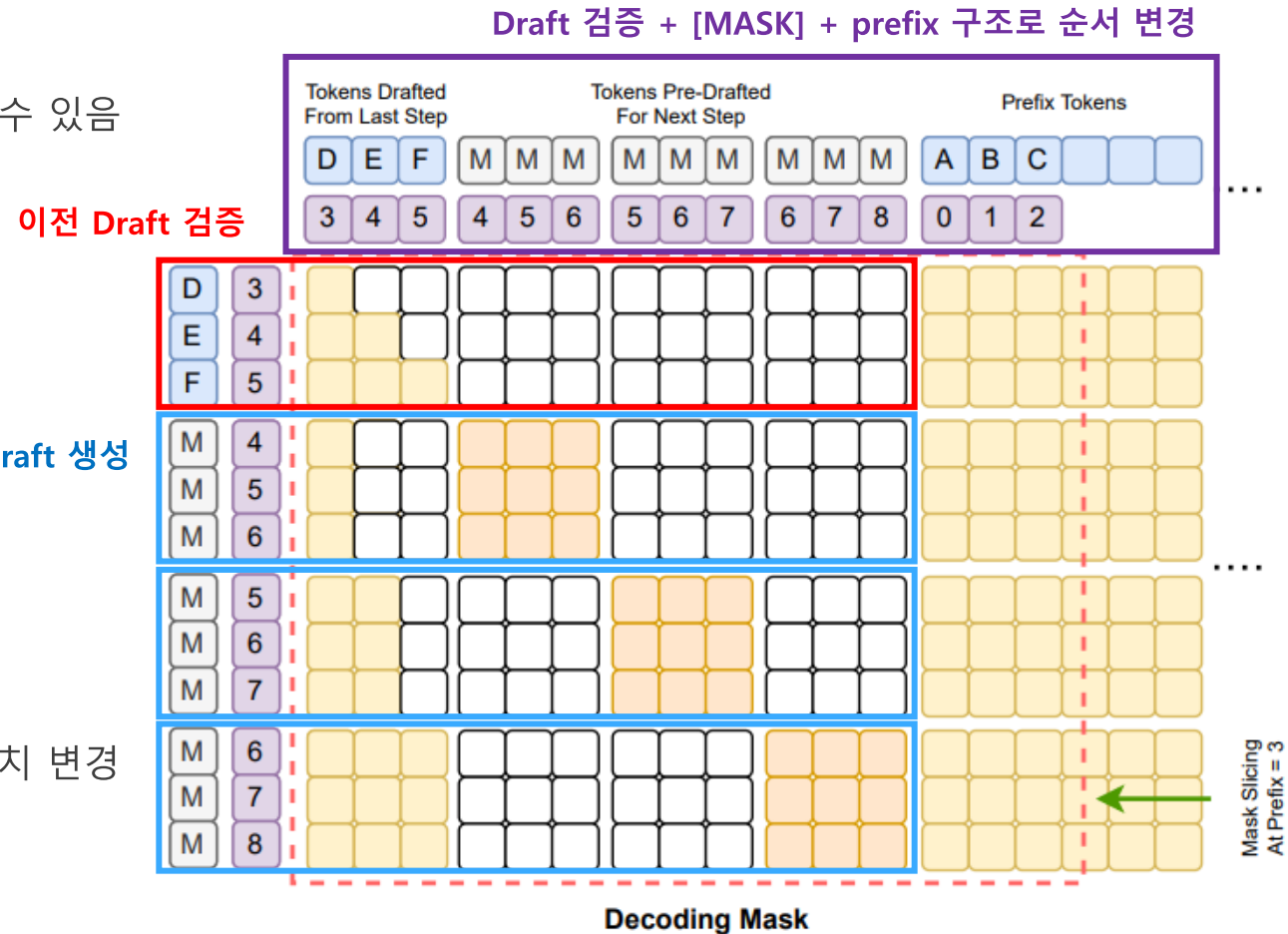
- 이전 timestep에서 결정된 draft token에 대해 검증
- Causal Attention 사용

One step Diffusion

- Block 내의 모든 token을 한번에 생성
- 가능한 모든 검증 결과에 대해서 미리 생성

Attention Map 수정

- Draft 검증 및 생성 부분과 prefix token의 물리적 위치 변경
- 다양한 길이의 input을 병렬처리에 유용
- 연산 결과는 변경 전과 동일



Optimization (Training & Inference)

Stable Training & Inference Optimization

Train 과정

- AR mode와 Diffusion mode가 학습되는 loss 함수 및 token의 수가 동일하기 때문에 loss scale의 분산이 유사함
- 따라서 두 loss 함수 사이에 비중을 조율하는 hyperparameter가 있긴 하지만 1로 설정함

Inference 과정

- 이전 draft token들의 검증과 동시에 다음 draft를 생성
- 확정된 token들은 causal attention을 따르기 때문에 KV Cache를 사용할 수 있음

Mode Aggregation

- 최종 token 생성에서 두 mode의 logit을 반영할 수 있음

$$\arg \max_i \{ \beta \text{ logits}_i^{ar} + (1 - \beta) \beta \text{ logits}_i^{diff}, i \in |V| \}$$

Experiments

Generation Evaluation Results

(괄호: forward 1번에 평균 생성 token 수)

Model Arch	Size	Coding				Math		Avg
		HumanEval	HumanEval+	MBPP	MBPP+	GSM8k	Minerva Math	
Llama3.2	1B	17.68%	14.63%	26.60%	38.89%	5.67%	1.92%	17.57%
SmolLM2	1.7B	0.61%	0.61%	35.40%	47.62%	28.20%	11.28%	20.62%
Qwen2.5	0.5B	27.44%	25.61%	29.60%	44.97%	37.45%	14.48%	29.92%
Qwen2.5	1.5B	35.98%	29.88%	43.60%	59.23%	54.74%	26.40%	41.64%
Qwen3	1.7B	48.17%	41.46%	55.80%	71.43%	66.72%	29.74%	52.22%
Block Diff	1.5B*	39.02%	34.76%	34.00%	48.15%	52.99%	21.56%	38.41%
TiDAR	1.5B*	43.29% (6.50)	39.02% (6.50)	41.40% (9.25)	61.11% (9.43)	53.90% (5.07)	25.48% (7.92)	44.03% (7.45)
Qwen3	4B	57.32%	50.61%	67.00%	80.69%	77.48%	47.10%	63.37%
Qwen3	8B	64.63%	56.71%	69.40%	83.07%	81.80%	52.94%	68.09%
LLaDA	8B	32.32%	27.44%	40.80%	51.85%	70.96%	27.30%	41.78%
Dream	7B	54.88%	49.39%	56.80%	74.60%	77.18%	39.60%	58.74%
Block Diff	4B [†]	56.10%	51.22%	54.60%	69.84%	82.87%	47.02%	60.27%
TiDAR (Trust AR)	8B [‡]	55.49% (7.46)	52.44% (7.44)	65.40% (9.96)	79.63% (10.13)	79.83% (6.90)	50.58% (7.48)	63.90% (8.23)
TiDAR (Trust Diff)	8B [‡]	57.93% (7.30)	55.49% (7.29)	65.40% (10.00)	80.95% (10.13)	80.44% (7.07)	51.64% (7.68)	65.31% (8.25)

Experiments

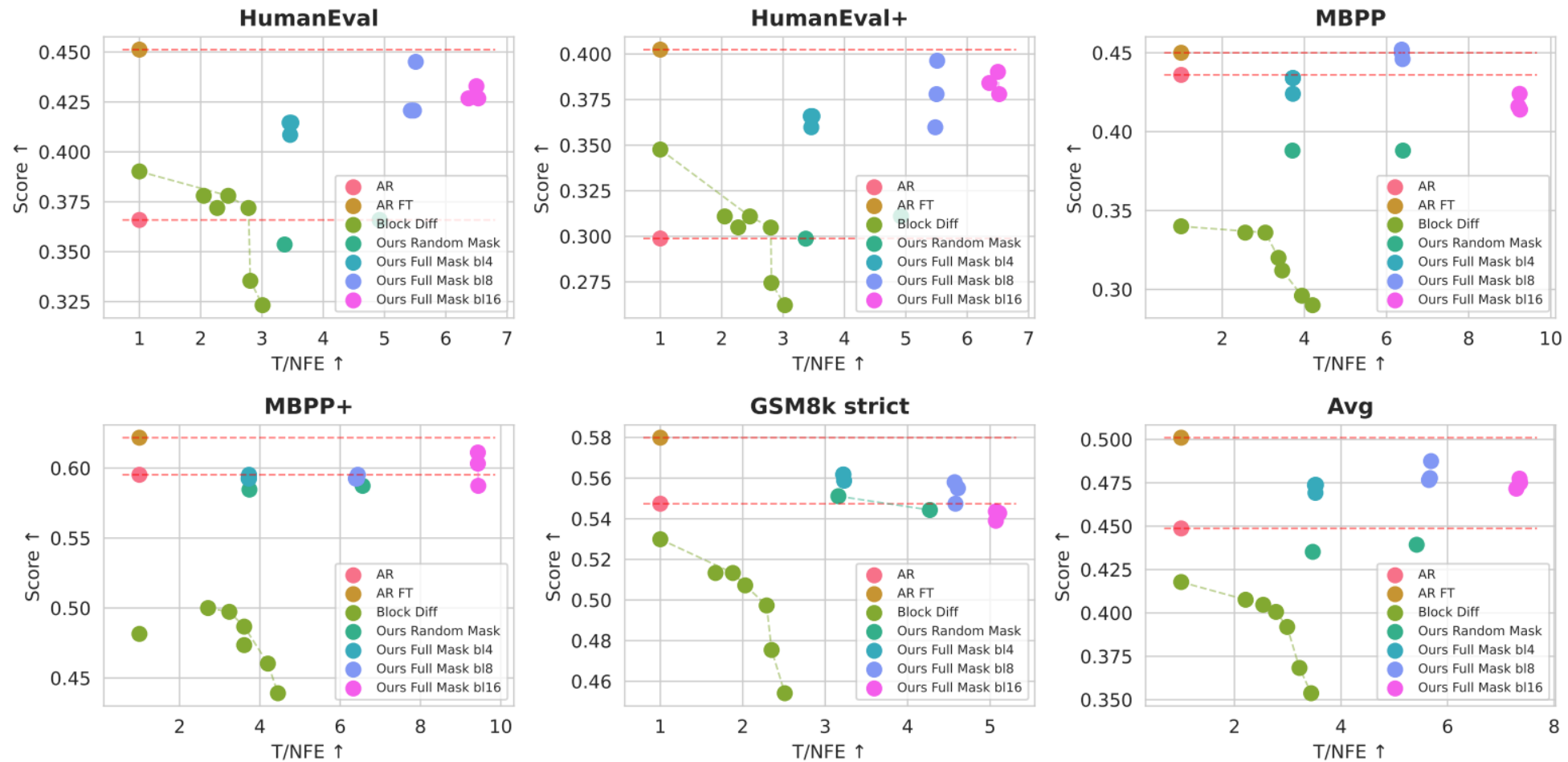
○ Likelihood Evaluation Results

Model Arch	Size	Knowledge		Commonsense Reasoning				
		MMLU	ARC-e	ARC-c	Hellaswag	PIQA	Winogrande	Avg
Llama3.2	1B	30.98%	65.28%	36.35%	63.76%	74.43%	63.30%	55.68%
SmolLM2	1.7B	49.99%	77.82%	47.44%	71.44%	77.64%	67.88%	65.37%
Qwen2.5	0.5B	47.65%	64.77%	31.83%	52.25%	70.02%	57.70%	54.04%
Qwen2.5	1.5B	60.96%	75.17%	45.05%	67.90%	76.12%	65.75%	65.16%
Qwen3	1.7B	62.53%	73.32%	44.62%	66.43%	75.63%	64.96%	64.58%
<i>Block Diff</i>	1.5B*	57.94%	74.41%	45.73%	56.26%	70.13%	61.80%	61.05%
TiDAR	1.5B*	58.99%	77.78%	45.39%	65.26%	75.52%	63.61%	64.43%
Qwen3	4B	73.00%	79.00%	52.00%	74.00%	78.00%	72.00%	71.33%
Qwen3	8B	76.93%	81.90%	53.16%	78.59%	79.22%	75.69%	74.25%
<i>Block Diff</i>	4B [†]	71.53%	81.48%	55.63%	65.48%	74.92%	70.96%	70.00%
<i>Dream</i>	7B	67.00%	82.20%	59.13%	73.73%	75.52%	73.56%	71.86%
LLaDA	8B	65.86%	73.78%	49.15%	71.05%	73.88%	74.66%	68.06%
TiDAR	8B [‡]	76.57%	84.18%	58.53%	76.36%	80.25%	76.48%	75.40%

Experiments

Performance-Efficiency trade-offs

(T/NFE = forward 연산 1번에 확정되는 평균 token 수)



CForce: Boosting Parallel Decoding for dLLMs via Consistency Forcing

Yuji Ren^{1,2*}, Chenkai Xu^{1,2*}, Zhuocheng Gong², Jianguo Li^{2†}, Zhijie Deng^{1†}

¹Shanghai Jiao Tong University ²Ant Group

{renyj26, 132435xck, zhijied}@sjtu.edu.cn, lijg.zero@antgroup.com

CForce: Boosting Parallel Decoding for dLLMs via Consistency Forcing

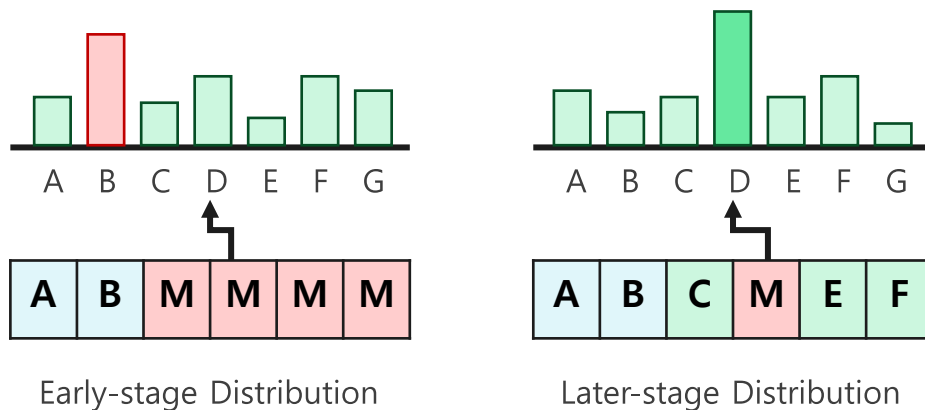
○ Motivation

○ Early-stage prediction의 신뢰도 문제

- 효율성을 위해 여러 token을 동시에 decoding하는 경우, 정보가 부족한 early-stage의 예측에 대한 신뢰도가 부족함
- Early-stage에서 error가 발생하면 이는 이후 stage로도 전파됨

○ Train-Test mismatch

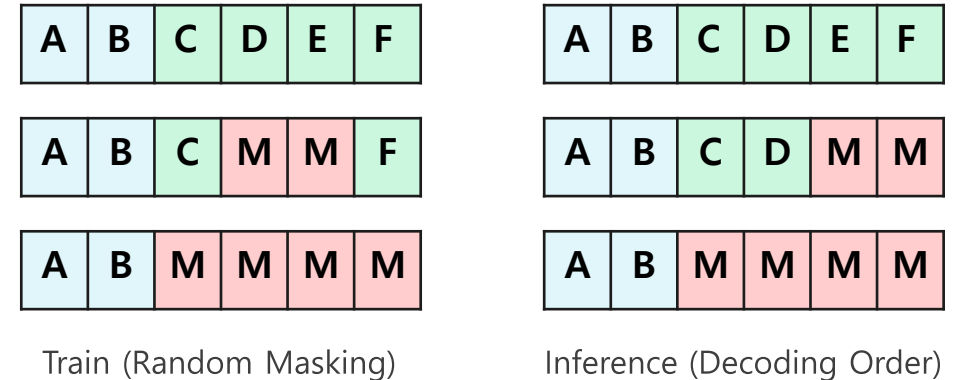
- 학습 과정에서 무작위로 masking된 sequence로부터 정답 token을 예측하도록 학습
- 하지만 실제 추론 단계에서는 모델 denoising 중인 text에 대해서 [MASK] token을 예측해야 함
- 이러한 학습, 추론 데이터 분포 차이가 성능 저하를 유발할 수 있음



Early-stage Distribution

Later-stage Distribution

<Early-stage Unreliable>



Train (Random Masking)

Inference (Decoding Order)

<Train-Test mismatch>

CForce: Boosting Parallel Decoding for dLLMs via Consistency Forcing

Overview of CForce

- 정보를 더 많이 갖는 상태와 Knowledge Distillation을 수행하자
- 모델이 실제 denoising하는 과정의 text를 학습 데이터로 사용하자

1. Collect Confidence-based Trajectory $\mathcal{T} = (s_0, s_1, \dots, s_T)$

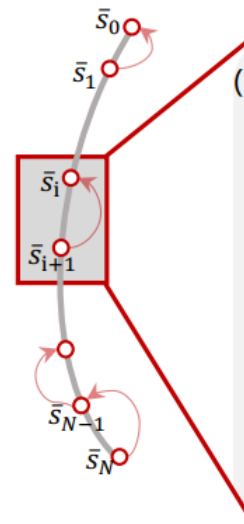
Step 0	[MASK]	[MASK]	[MASK]	[MASK]	[MASK]	[MASK]	[MASK]
Step 1	Today	is	[MASK]	[MASK]	[MASK]	[MASK]	[MASK]
Step 2	Today	is	good	day	[MASK]	[MASK]	[MASK]
Step 3	Today	is	a	good	[MASK]	[MASK]	dLLM
Step 4	Today	is	a	good	[MASK]	to	study
Step 5	Today	is	a	good	day	to	study

2. Construct Staged Trajectory $\bar{\mathcal{T}} = (\bar{s}_0, \bar{s}_1, \dots, \bar{s}_N)$

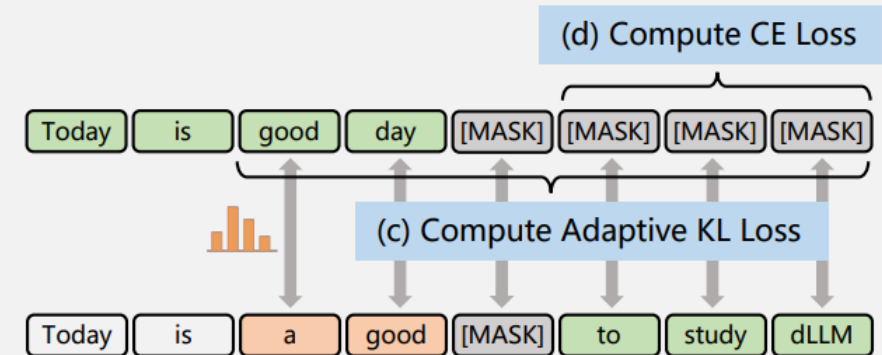
Stage 0	[MASK]	[MASK]	[MASK]	[MASK]	[MASK]	[MASK]	[MASK]
Stage 1	Today	is	good	day	[MASK]	[MASK]	[MASK]
Stage 2	Today	is	a	good	[MASK]	to	study
Stage 3	Today	is	a	good	day	to	study

3. Train with Adjacent-stage Consistency Forcing

(a) Sample an adjacent pair of stages $(\bar{s}_i, \bar{s}_{i+1})$



(b) $\bar{s}_{i+1}$ plays as the target, $\bar{s}_i$ as the student



Training CForce

Staged Trajectory Construction

Collect Trajectory

- 학습할 Target Model을 통해 Trajectory를 수집 (Confidence-based)

$$\mathcal{T} = (s_0, s_1, \dots, s_T)$$

Staged Trajectory 구축

- [MASK] token이 reveal되는 수를 기준으로 staged trajectory를 구축

$$\bar{\mathcal{T}} = (\bar{s}_0, \bar{s}_1, \dots, \bar{s}_N)$$

- 각 stage에 대하여 masked position set $\bar{M}_i = \{j: \bar{s}_i^j = [MASK]\}$ 정의
- 인접한 두 stage 쌍 $(\bar{s}_i, \bar{s}_{i+1})$ 에 대하여 [MASK]가 유지되는 position set $\mathcal{U}_i$ 과 reveal 되는 position set Δ_i 을 정의

$$\mathcal{U}_i = \bar{M}_i \cap \bar{M}_{i+1},$$

$$\Delta_i = \bar{M}_i \setminus \bar{M}_{i+1}$$

1. Collect Confidence-based Trajectory $\mathcal{T} = (s_0, s_1, \dots, s_T)$

Step 0	[MASK]	[MASK]	[MASK]	[MASK]	[MASK]	[MASK]	[MASK]	[MASK]
Step 1	Today	is	[MASK]	[MASK]	[MASK]	[MASK]	[MASK]	[MASK]
Step 2	Today	is	good	day	[MASK]	[MASK]	[MASK]	[MASK]
Step 3	Today	is	a	good	[MASK]	[MASK]	[MASK]	dLLM
Step 4	Today	is	a	good	[MASK]	to	study	dLLM
Step 5	Today	is	a	good	day	to	study	dLLM

2. Construct Staged Trajectory $\bar{\mathcal{T}} = (\bar{s}_0, \bar{s}_1, \dots, \bar{s}_N)$

Stage 0	[MASK]	[MASK]	[MASK]	[MASK]	[MASK]	[MASK]	[MASK]	[MASK]
Stage 1	Today	is	good	day	[MASK]	[MASK]	[MASK]	[MASK]
Stage 2	Today	is	a	good	[MASK]	to	study	dLLM
Stage 3	Today	is	a	good	day	to	study	dLLM

Training CForce

Adjacent-stage Consistency Forcing

Confidence Adaptive KL Divergence

- Adjacent stage pair $(\bar{s}_i, \bar{s}_{i+1})$ 에서 두 분포를 정의

$$p_{stu}^j = q_\theta(\cdot | x, \bar{s}_i, j), \quad p_{tar}^j = sg[q_\theta(\cdot | x, \bar{s}_{i+1}, j)]$$

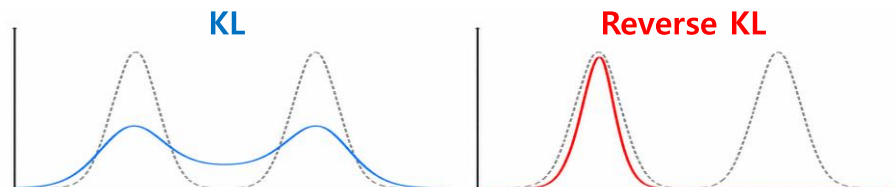
- 두 stage 모두에서 [MASK]인 position set $\mathcal{U}_i$ 에 대하여

$$c_j = \max_v p_{tar}^j(v)$$

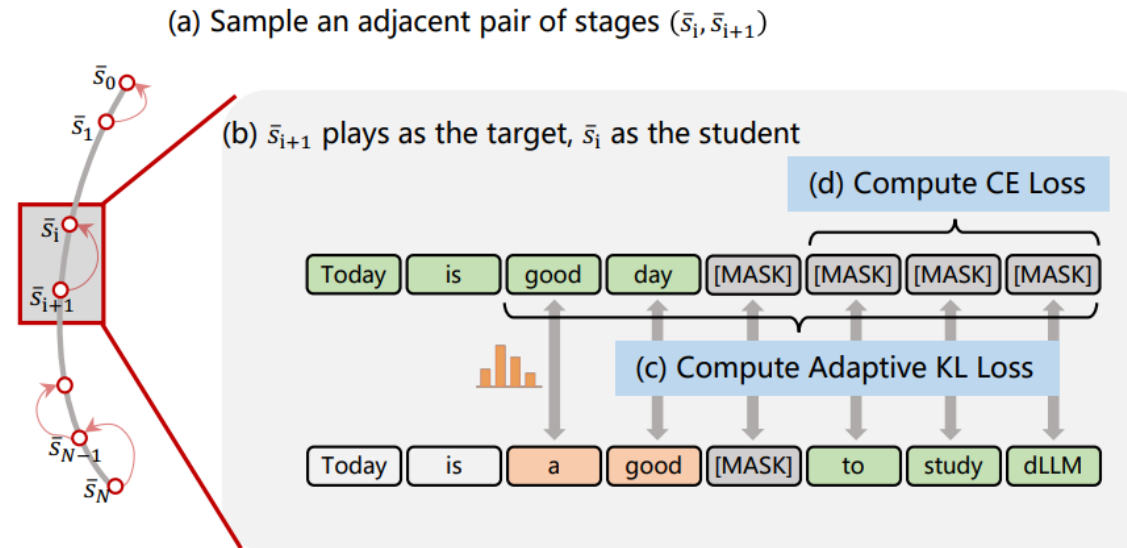
$$\mathcal{L}_{CAD} = \frac{1}{|\mathcal{U}_i|} \sum_{j \in \mathcal{U}_i} [D_{KL}(p_{tar}^j || p_{stu}^j) + c_j D_{KL}(p_{stu}^j || p_{tar}^j)]$$

Why Confidence Adaptive KL?

- KL Divergence는 target의 분포를 따라가게 학습하지만, 분포가 뭉개지는 경향이 있음
- 반대로 Reverse KL은 sharp한 분포를 학습하지만, mode seeking의 경향이 있음
- 따라서 이 2가지 성향을 모두 가지되, 불확실한 분포에서 mode seeking이 발생하지 않도록 confidence를 적용



3. Train with Adjacent-stage Consistency Forcing



Training CForce

Adjacent-stage Consistency Forcing

CE Anchor

- 두 stage 사이에서 reveal되는 position set Δ_i 에 대하여

$$\mathcal{L}_{CE} = -\frac{1}{|\Delta_i|} \sum_{j \in \Delta_i} \log q_{\theta}(y^j | x, \bar{s}_i, j)$$

- CE Loss는 token이 확정되기 직전의 확률 분포에 적용
- 잘못된 token 확정으로 error가 전파되는 것을 방지

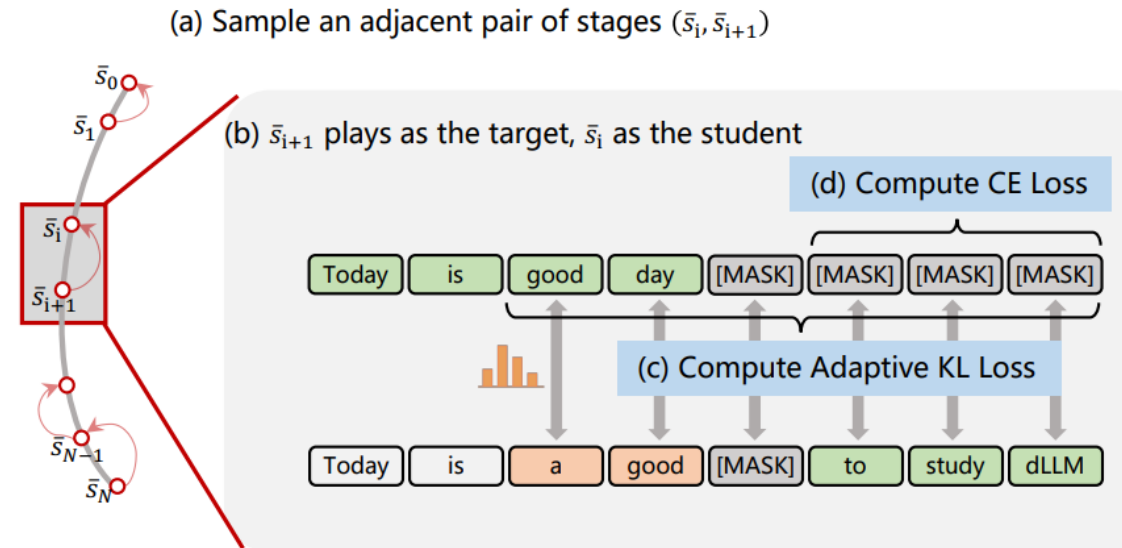
Curriculum Transition

초기 학습에는 reveal token의 일부만 학습

- Training step u 에서 ρ_u 의 token들만 target context에 추가
- 나머지는 reveal token들은 reveal되지 않은 것으로 취급

$$\rho_u = \min\left(1, \rho_0 + (1 - \rho_0) \frac{u}{T_{train}}\right)$$

3. Train with Adjacent-stage Consistency Forcing



Training CForce

○ CForce with Editable DLM

○ Staged Trajectory of Editable DLM

- 이미 확정한 token도 다른 token으로 수정하는 DLM
- CForce의 Staged Trajectory에서 token이 수정되는 경우가 존재할 수 있음
- Stage-level edit set $\bar{E}_i$

$$\bar{E}_i = \{j: \bar{s}_i \neq [\text{MASK}], \bar{s}_{j+1} \neq [\text{MASK}], \bar{s}_j \neq \bar{s}_{j+1}\}$$

○ CAD of Editable DLM

- CAD loss의 학습 token을 [MASK] token과 이후 수정될 token으로 확장 ($\mathcal{U}_i \rightarrow \bar{\mathcal{M}}_i \cup \bar{E}_i$)
- Editable DLM에서는 token이 결정되어도 이후에 수정될 수 있기 때문에 학습 과정에서 [MASK] token처럼 취급하는 것

○ CE of Editable DLM

- CE loss의 학습 token을 [MASK] token으로 확장 ($\Delta_i \rightarrow \bar{\mathcal{M}}_i$)
- CE loss는 다음 stage의 token이 아니라, 마지막 stage의 token
- 초기 단계에서 올바른 예측을 유도하려는 의도

Training CForce

Theoretical Analysis

- Total variance distance $D_{TV}(p, q)$

$$D_{TV}(p, q) = \frac{1}{2} \sum_{v \in \mathcal{V}} |p(v) - q(v)|$$

- CAD와 CE가 어떻게 정답 token의 one-hot vector δ_{yj} 과 현재 stage $\bar{s}_i$ 의 $D_{TV}(q_\theta(\cdot | x, \bar{s}_i, j), \delta_{yj})$ 를 줄이는가?

$q_\theta(\cdot | x, \bar{s}_i, j)$ 를 p_i^j , j position이 reveal되는 stage를 r_j 라고 하면 삼각부등식에 의해 다음이 성립함

$$D_{TV}(p_i^j, \delta_{yj}) \leq \sum_{r=i}^{r_j-2} \mathbf{D}_{TV}(\mathbf{p}_r^j, \mathbf{p}_{r+1}^j) + \mathbf{D}_{TV}(\mathbf{p}_{r_j-1}^j, \delta_{yj})$$

Pinsker's inequality에 의해서 adjacent-stage TV term과 forward KL 사이에 다음이 성립함

$$\mathbf{D}_{TV}(\mathbf{p}_r^j, \mathbf{p}_{r+1}^j) \leq \sqrt{\frac{1}{2} D_{KL}(p_{r+1}^j || p_r^j)}$$

CAD loss는 KL divergence의 upper bound임

$$\mathcal{L}_{CAD}^j = D_{KL}(p_{tar}^j || p_{stu}^j) + c_j D_{KL}(p_{stu}^j || p_{tar}^j) \geq D_{KL}(p_{tar}^j || p_{stu}^j)$$

Training CForce

Theoretical Analysis

- CAD와 CE가 어떻게 정답 token의 one-hot vector δ_{y^j} 과 현재 stage $\bar{s}_i$ 의 $D_{TV}(q_\theta(\cdot | x, \bar{s}_i, j), \delta_{y^j})$ 를 줄이는가?

$$D_{TV}(p_i^j, \delta_{y^j}) \leq \sum_{r=i}^{r_j-2} D_{TV}(p_r^j, p_{r+1}^j) + D_{TV}(p_{r_j-1}^j, \delta_{y^j})$$

위 식의 두번째 항은 negative log likelihood로 표현할 수 있음

$$D_{TV}(p_{r_j-1}^j, \delta_{y^j}) = 1 - p_{r_j-1}^j(y^j) \leq -\log p_{r_j-1}^j(y^j)$$

CE loss는 $D_{TV}(p_{r_j-1}^j, \delta_{y^j})$ 의 upper bound임

따라서 본 논문의 최종 objective function은 현재 상태에서의 모델 분포와 정답 token 분포의 차이를 줄이도록 학습

Experiments

Main Comparison

Diffusion type	Edit?	Model	GSM8K			MATH500			MBPP			HumanEval		
			Score	TPF	AUP	Score	TPF	AUP	Score	TPF	AUP	Score	TPF	AUP
Pure diffusion	×	dUltra-coding-b32	81.52 [*]	8.40	—	35.64	6.72 [*]	—	37.04	7.29 [*]	—	35.85	7.23 [*]	—
		d3LLM-LLaDA	73.09	9.11 [*]	637.65 [*]	30.36	5.74	107.64 [*]	40.60	4.21	88.36	39.63	5.95	96.64
		d3LLM-Dream	81.36	4.94	391.33	38.21 [*]	3.92	97.50	55.60 [*]	2.96	141.41 [*]	57.10 [*]	3.20	129.48 [*]
		D2F-LLaDA	74.39	2.88	213.76	28.94	2.66	49.00	39.00	2.13	52.96	40.64	2.69	61.98
Block diffusion	×	LightningRL-8B-b32	90.30	5.58	492.40	63.00	<u>6.28</u>	<u>407.50</u>	58.30	11.10	641.60	72.60	<u>6.30</u>	<u>450.10</u>
		LLaDA2.0-mini	93.25	2.85	266.82	81.80	3.46	286.71	78.69	3.80	305.84	83.54	4.28	367.52
		LLaDA2.0-mini-CAP	<u>91.74</u>	4.09	374.03	<u>81.00</u>	4.85	394.10	72.13	5.43	<u>400.00</u>	73.18	5.89	387.79
		LLaDA2.0-mini-CForce (Ours)	<u>91.74</u>	<u>5.36</u>	<u>489.07</u>	79.20	6.35	509.23	<u>73.30</u>	<u>6.00</u>	387.80	<u>79.27</u>	7.97	625.51
	✓	DMax	92.10	5.48	557.00	75.40	5.94	507.00	<u>79.20</u>	5.86	482.00	83.50	<u>7.36</u>	637.00
		LLaDA2.1-mini	93.56	<u>5.94</u>	<u>666.71</u>	85.00	<u>7.44</u>	<u>741.42</u>	<u>77.75</u>	<u>7.25</u>	<u>636.16</u>	<u>85.98</u>	7.11	<u>705.47</u>
		LLaDA2.1-mini-CForce (Ours)	<u>92.27</u>	7.63	845.03	<u>84.80</u>	10.15	966.35	81.97	10.07	1016.14	86.59	8.48	870.90

Experiments

○ Ablation Study

- **Frozen Teacher:** target 분포 = pretrained model의 logit
- **Stage size:** stage 사이에 reveal되는 token 수

Training		AVG	
Curriculum	Frozen Teacher	Score	TPF
✓		80.88	6.42
✓	✓	79.49	5.91
		79.87	5.97

Table 3: **Ablation on Curriculum Learning and Target Type for LLaDA2.0-mini-CForce.** The frozen-teacher variant changes only the target predictor; the no-curriculum variant removes the gradual exposure schedule from the same-model stop-gradient target.

Stage size S	AVG Score	AVG TPF
4	80.62	5.64
8	80.88	6.42
16	75.05	7.97

Table 2: **Ablation on Stage Size for LLaDA2.0-mini-CForce.** Full benchmark-level results are in Appendix F. The shaded row denotes the default setting; bold marks the best value in each metric.

Component	Variant	AVG	
		Score	TPF
KL divergence	Forward KL	85.75	5.07
	Reverse KL	65.57	8.72
	CAD	80.88	6.42
CE anchor	$\lambda_{CE} = 0$	79.02	6.52
	$\lambda_{CE} = 0.1$	80.88	6.42
	$\lambda_{CE} = 2.0$	80.22	6.52

Table 4: **Ablations of the KL Divergence Form and CE Anchor Weight on LLaDA2.0-mini-CForce.**

Other Research Topics

○ 세미나에서 다루지 않은 다른 연구 주제들

○ Continuous Diffusion Language Modeling

- Masked Diffusion Language Modeling은 수학적으로 Diffusion 다름
- 최근 MDLM의 발전에 따라 Continuous DLM 연구도 다시 수행됨

○ Decoding Strategy

- 어떤 방식으로 생성 token을 결정할 것인가?
- Confidence, Stability, Entropy or ???

○ Reinforcement Learning on DLM

- 기존의 AR LLM에서 강화학습은 큰 성능 향상 효과를 보임
- DLM에서는 RL을 어떻게 적용할 것인가?



Thank you

Dooyoung Kim
(kdysunleo98@gmail.com)



질문 <https://forms.gle/LtvyMJ7BFwMKpWtz8>

피드백 <https://forms.gle/PAmxLQnRBZVhAMaw8>

응답 <https://docs.google.com/spreadsheets/d/1uWyc0pUfQOwTTZUDyY3gxImU5xcFro90kKJKOcDitnk/edit?usp=sharing>